

CO3-AT3-Part2

EXPERIMENT

#=====

ACTOR-CRITICMETHODS (A2C, A3C)

Name:N.Akshaya

Reg.no:192425129

Course code:MLA0305

Course name: REINFORCEMENT LEARNING

Slot: D

Aim:

To implement and compare **Advantage Actor-Critic (A2C)** and **Asynchronous Advantage Actor-Critic (A3C)** methods for reinforcement learning, and visualize their learning performance using colored graphs.

Procedure:

1. Create a reinforcement learning environment using Gymnasium.
2. Define an Actor-Critic neural network containing:
 - **Actor:** predicts the probability of each action.
 - **Critic:** estimates the value of the current state.
3. Implement the **A2C** algorithm using multiple episodes.
4. Implement an **A3C-style** parallel training approach using multiple workers.
5. Store the total reward obtained in every episode.
6. Calculate the average reward for both methods.
7. Compare A2C and A3C using:
 - Episode reward graph

- Moving-average reward graph
- Final performance bar chart

8. Display a summary of the training results.

Python Code:

```
# EXPERIMENT: ACTOR-CRITIC METHODS - A2C AND A3C
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import gymnasium as gym
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
from torch.distributions import Categorical
```

```
from multiprocessing import Process, Queue
```

```
# SETTINGS
```

```
ENV_NAME = "CartPole-v1"
```

```
GAMMA = 0.99
```

```
LEARNING_RATE = 0.001
```

```
A2C_EPISODES = 300
```

```
A3C_EPISODES = 300
```

```
SEED = 42
```

```
np.random.seed(SEED)
```

```
torch.manual_seed(SEED)
```

```
# ACTOR-CRITIC NETWORK
```

```
class ActorCritic(nn.Module):
```

```

def __init__(self, state_size, action_size):

    super(ActorCritic, self).__init__()

    self.shared = nn.Sequential(

        nn.Linear(state_size, 128),

        nn.ReLU()

    )

    # Actor

    self.actor = nn.Sequential(

        nn.Linear(128, action_size),

        nn.Softmax(dim=-1)

    )

    # Critic

    self.critic = nn.Linear(128, 1)

def forward(self, state):

    x = self.shared(state)

    action_probs = self.actor(x)

    value = self.critic(x)

    return action_probs, value

# SELECT ACTION

def select_action(model, state):

    state = torch.FloatTensor(state).unsqueeze(0)

    probs, value = model(state)

```

```

distribution = Categorical(probs)

action = distribution.sample()

log_prob = distribution.log_prob(action)

entropy = distribution.entropy()

return action.item(), log_prob, value.squeeze(), entropy

# A2C IMPLEMENTATION

def train_a2c():

    env = gym.make(ENV_NAME)

    state_size = env.observation_space.shape[0]

    action_size = env.action_space.n

    model = ActorCritic(state_size, action_size)

    optimizer = optim.Adam(

        model.parameters(),

        lr=LEARNING_RATE

    )

    rewards_history = []

    for episode in range(A2C_EPISODES):

        state, _ = env.reset(seed=SEED + episode)

        log_probs = []

        values = []

        rewards = []

        entropies = []

        done = False

```

while not done:

```
    action, log_prob, value, entropy = \
        select_action(model, state)

    next_state, reward, terminated, truncated, _ = \
        env.step(action)

    done = terminated or truncated

    log_probs.append(log_prob)
    values.append(value)
    rewards.append(reward)
    entropies.append(entropy)

    state = next_state

# Calculate Return
returns = []

G = 0

for reward in reversed(rewards):

    G = reward + GAMMA * G

    returns.insert(0, G)

returns = torch.FloatTensor(returns)

values_tensor = torch.stack(values)

log_probs_tensor = torch.stack(log_probs)

entropy_tensor = torch.stack(entropies)

advantages = returns - values_tensor.detach()

actor_loss = -(log_probs_tensor * advantages).mean()
```

```

critic_loss = \
    nn.functional.mse_loss(
        values_tensor,
        returns
    )
entropy_loss = -0.01 * entropy_tensor.mean()
loss = actor_loss + 0.5 * critic_loss + entropy_loss
optimizer.zero_grad()
loss.backward()
optimizer.step()
total_reward = sum(rewards)
rewards_history.append(total_reward)
if (episode + 1) % 50 == 0:
    avg_reward = np.mean(
        rewards_history[-50:]
    )
    print(
        f"A2C Episode {episode + 1:3d} | "
        f"Average Reward: {avg_reward:.2f}"
    )
env.close()
return rewards_history

```

```

# A3C-STYLE PARALLEL IMPLEMENTATION

def train_worker(worker_id, episodes, queue):

    env = gym.make(ENV_NAME)

    state_size = env.observation_space.shape[0]

    action_size = env.action_space.n

    model = ActorCritic(state_size, action_size)

    optimizer = optim.Adam(

        model.parameters(),

        lr=LEARNING_RATE

    )

    rewards_history = []

    for episode in range(episodes):

        state, _ = env.reset(

            seed=SEED + worker_id * 1000 + episode

        )

        log_probs = []

        values = []

        rewards = []

        entropies = []

        done = False

        while not done:

            action, log_prob, value, entropy = \

                select_action(model, state)

```

```

next_state, reward, terminated, truncated, _ = \
    env.step(action)

done = terminated or truncated

log_probs.append(log_prob)

values.append(value)

rewards.append(reward)

entropies.append(entropy)

state = next_state

# Calculate Return

returns = []

G = 0

for reward in reversed(rewards):

    G = reward + GAMMA * G

    returns.insert(0, G)

returns = torch.FloatTensor(returns)

values_tensor = torch.stack(values)

log_probs_tensor = torch.stack(log_probs)

entropy_tensor = torch.stack(entropies)

advantages = returns - values_tensor.detach()

actor_loss = -(log_probs_tensor * advantages).mean()

critic_loss = \

    nn.functional.mse_loss(

        values_tensor,

        returns

```

```

    )

    entropy_loss = -0.01 * entropy_tensor.mean()

    loss = actor_loss + 0.5 * critic_loss + entropy_loss

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    total_reward = sum(rewards)

    rewards_history.append(total_reward)

env.close()

queue.put(rewards_history)

# A3C TRAINING

def train_a3c():

    workers = 4

    episodes_per_worker = A3C_EPISODES // workers

    queue = Queue()

    processes = []

    for worker in range(workers):

        process = Process(

            target=train_worker,

            args=(

                worker,

                episodes_per_worker,

                queue

            )

```

```

    )

    process.start()

    processes.append(process)

all_rewards = []

for _ in range(workers):

    rewards = queue.get()

    all_rewards.extend(rewards)

for process in processes:

    process.join()

return all_rewards

# RUN A2C

print("\n" + "=" * 60)

print("TRAINING A2C")

print("=" * 60)

a2c_rewards = train_a2c()

# RUN A3C

print("\n" + "=" * 60)

print("TRAINING A3C")

print("=" * 60)

a3c_rewards = train_a3c()

# MOVING AVERAGE FUNCTION

def moving_average(data, window=20):

    return np.convolve(

        data,

```

```

        np.ones(window) / window,

        mode="valid"

    )

a2c_avg = moving_average(a2c_rewards)

a3c_avg = moving_average(a3c_rewards)

# SUMMARY OUTPUT

print("\n")

print("=" * 60)

print("          TRAINING SUMMARY")

print("=" * 60)

print(f"Number of A2C Episodes : {len(a2c_rewards)}")

print(f"Number of A3C Episodes : {len(a3c_rewards)}")

print(

    f"\nA2C Average Reward   : "

    f"{np.mean(a2c_rewards):.2f}"

)

print(

    f"A3C Average Reward   : "

    f"{np.mean(a3c_rewards):.2f}"

)

print(

    f"\nA2C Final 50 Average : "

    f"{np.mean(a2c_rewards[-50:]):.2f}"

)

```

```

print(
    f"A3C Final 50 Average : "
    f"{np.mean(a3c_rewards[-50:]):.2f}"
)

print(
    f"\nA2C Maximum Reward : "
    f"{np.max(a2c_rewards):.2f}"
)

print(
    f"A3C Maximum Reward : "
    f"{np.max(a3c_rewards):.2f}"
)

if np.mean(a2c_rewards[-50:]) > \
    np.mean(a3c_rewards[-50:]):
    print("\nBetter Final Performance : A2C")
else:
    print("\nBetter Final Performance : A3C")

print("=" * 60)

# VISUALIZATION 1

# EPISODE REWARD COMPARISON

plt.figure(figsize=(12, 6))

plt.plot(
    a2c_rewards,

```

```
        label="A2C Reward",
        color="blue",
        alpha=0.55
    )
plt.plot(
    a3c_rewards,
    label="A3C Reward",
    color="red",
    alpha=0.55
)
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.title(
    "A2C vs A3C - Episode Reward Comparison"
)
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# VISUALIZATION 2

# MOVING AVERAGE

plt.figure(figsize=(12, 6))

plt.plot(
    a2c_avg,
```

```
    label="A2C Moving Average",
    color="green",
    linewidth=2.5
)

plt.plot(
    a3c_avg,
    label="A3C Moving Average",
    color="purple",
    linewidth=2.5
)

plt.xlabel("Episode")
plt.ylabel("Average Reward")
plt.title(
    "A2C vs A3C - Learning Curve"
)

plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# VISUALIZATION 3

# FINAL PERFORMANCE BAR CHART

a2c_final = np.mean(a2c_rewards[-50:])
a3c_final = np.mean(a3c_rewards[-50:])
```

```
methods = ["A2C", "A3C"]

performance = [a2c_final, a3c_final]

plt.figure(figsize=(8, 6))

bars = plt.bar(

    methods,

    performance,

    color=["orange", "teal"],

    edgecolor="black",

    linewidth=1.2

)

plt.ylabel("Average Reward")

plt.title(

    "Final Performance Comparison"

)

plt.grid(

    axis="y",

    alpha=0.3

)

for bar in bars:

    height = bar.get_height()

    plt.text(

        bar.get_x() + bar.get_width() / 2,

        height + 1,

        f"{height:.2f}",
```

```
        ha="center",

        fontsize=11,

        fontweight="bold"

    )

plt.tight_layout()

plt.show()

# VISUALIZATION 4

# REWARD DISTRIBUTION

plt.figure(figsize=(10, 6))

plt.hist(

    a2c_rewards,

    bins=20,

    alpha=0.65,

    label="A2C",

    color="blue",

    edgecolor="black"

)

plt.hist(

    a3c_rewards,

    bins=20,

    alpha=0.65,

    label="A3C",

    color="red",

    edgecolor="black"
```

```

)

plt.xlabel("Reward")

plt.ylabel("Frequency")

plt.title(

    "Reward Distribution - A2C vs A3C"

)

plt.legend()

plt.grid(alpha=0.25)

plt.tight_layout()

plt.show()

# FINAL RESULT

print("\n")

print("=" * 60)

print("          RESULT")

print("=" * 60)

print(

    "A2C and A3C Actor-Critic methods were "

    "successfully implemented."

)

print(

    "The learning curves show the change in "

    "reward during training."

)

```

```

print(

    "The final performance graph provides a "

    "direct comparison of A2C and A3C."

)

print("=" * 60)

```

Result:

A2C and A3C Actor–Critic methods were successfully implemented and compared on the CartPole environment. The reward curves and performance graphs provide a visual comparison of their learning behavior and final performance.

Summary Output:

```

=====
TRAINING A2C
=====
A2C Episode  50 | Average Reward: 18.56
A2C Episode 100 | Average Reward: 16.88
A2C Episode 150 | Average Reward: 17.38
A2C Episode 200 | Average Reward: 15.66
A2C Episode 250 | Average Reward: 18.52
A2C Episode 300 | Average Reward: 20.88

=====

TRAINING A3C
=====

=====
                        TRAINING SUMMARY
=====
Number of A2C Episodes : 300
Number of A3C Episodes : 300

A2C Average Reward      : 17.98
A3C Average Reward      : 18.93

A2C Final 50 Average    : 20.88
A3C Final 50 Average    : 19.16

A2C Maximum Reward      : 53.00
A3C Maximum Reward      : 52.00

Better Final Performance : A2C
=====

```

Visualization Output:



